

06 常见坑点速查

目标:Java 刷题 15 大经典坑,救命文档。做题前扫一遍,做题时遇到立刻查。

原则:每条坑都配"错误示例 + 正确写法 + 原因",不只说"不选 X"。

C++ 选手必看:有些坑 C++ 没有(如 Stack 过时、Integer ==),有些坑 C++ 也有(如浮点精度),但语义可能不同。文中标【C++ 选手必看】或【C++ 选手第 X 坑】的 5 个坑(坑 3 / 11 / 12 / 13 / 14)是 C++ 转 Java 第一周必踩。

一、序言:这一章怎么用

Java 刷题前 1-2 周,大部分调试时间不是花在算法上,而是花在 Java 语法和容器 API 的小细节上——一个 == 写错、一个 Stack 用错,代码就在 LeetCode 上 Wrong Answer 或者牛客上编译失败。

15 大坑按"必踩程度"排序,前 3 个 0 是第一周就会踩到的,中间 7 个 0 是做题 1-2 月内会踩到的,最后 5 个 0 是遇到特定场景才会踩。强烈建议:

- 第一次读:从前往后过一遍,代码块自己抄一次
- 做题前:扫一眼"避坑金句速记"
- 做题中:遇到报错/异常,翻对应坑点

二、15 大坑(按必踩程度排序)

坑 1: `Stack` 用 `ArrayDeque` 替(第一坑)

表现:用 java.util.Stack 性能差且过时,刷题时容易与方法名混淆。

原因:Stack 是 JDK 1.0 时代的设计,继承自 `Vector`,导致所有方法都带 synchronized 同步开销(单线程刷题完全没必要)。而且它同时支持栈语义(`push`/`pop`/`peek`)和 `Vector` 语义(`add`/`remove`),两种方法名混用容易出错。

错误:

```
import java.util.Stack;

Stack<Integer> s = new Stack<>();
s.push(1);
s.add(99);    // 编译能过,但 add 是 Vector 的方法,语义混乱
s.pop();
```

正确:

```
import java.util.ArrayDeque;
import java.util.Deque;

Deque<Integer> s = new ArrayDeque<>(); // 接口类型,栈/队列通用
s.push(1);
s.pop();
s.peek();
```

详见:03 容器与 STL 对照 第五节"深讲 2:ArrayDeque"。

C++ 对照:C++ std::stack 性能 OK(只是 deque 的薄包装),Java 这个坑是 Java 特有的容器设计问题。

坑 2: `PriorityQueue` 默认小顶堆(反的!)

表现:写完代码,期望 `peek()` 取到最大值,实际取到最小值——典型"代码对、答案错"。

原因:Java `PriorityQueue<E>` 默认小顶堆(堆顶是最小元素),C++ `std::priority_queue` 默认大顶堆(堆顶是最大元素)。完全相反。这是 C++ 选手转 Java 第一周 100% 踩一次的坑。

错误:

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.offer(3);
pq.offer(1);
pq.offer(2);
int top = pq.peek(); // 期望 3(大顶堆),实际 1(小顶堆)
```

正确:

```
// 大顶堆:反转比较器
PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a);
maxHeap.offer(3);
maxHeap.offer(1);
maxHeap.offer(2);
int top = maxHeap.peek(); // 3,符合 C++ 选手直觉

// 标准写法
PriorityQueue<Integer> maxHeap2 = new PriorityQueue<>(Comparator.reverseOrder());
```

详见:03 容器与 STL 对照 第六节"深讲 3:PriorityQueue"。

C++ 对照:

```
// C++ 默认大顶堆
std::priority_queue<int> pq;
pq.push(3); pq.push(1); pq.push(2);
pq.top(); // 3

// C++ 小顶堆要加 greater
std::priority_queue<int, std::vector<int>, std::greater<int>>> minPq;
```

记忆口诀:Java 小,C++ 大,反着来!

坑 3: `Integer` `==` 陷阱(缓存池)【C++ 选手必看】

表现:`Integer a = 200; Integer b = 200; if (a == b) { ... }` 这段代码,有时 `true`、有时 `false`,一脸懵。

原因:Java 自动装箱(`int` → `Integer`)时,-128 ~ 127 范围内的值会复用缓存池的同一个对象,这个范围外才会 `new Integer(x)`。比较引用时,缓存池内的相等,缓存池外的不等。

错误:

```
Integer a = 100, b = 100;
System.out.println(a == b); // true(在缓存池内,同一个对象)

Integer c = 200, d = 200;
System.out.println(c == d); // false(超出缓存池,new 出来两个对象)
```

正确:



```
Integer c = 200, d = 200;
if (c.equals(d)) { // true,永远用 equals 比内容
    // ...
}

// 或者先拆箱成 int 再比
if (c.intValue() == d.intValue()) {
    // ...
}
```

详见:03 容器与 STL 对照 第三节"装箱小框"。

C++ 对照:

```
// C++ 没有这个问题,int 是基本类型,== 就是值比较
int a = 200, b = 200;
cout << (a == b); // true,无误
```

记忆口诀:Integer / Long / Double / String 比内容,一律 `.equals()`。

坑 4:遍历集合时 `remove` 抛 `ConcurrentModificationException`()

表现:for (Integer x : list) { if (x == 1) list.remove(x); } 运行时抛 ConcurrentModificationException,IDE 不报错但提交就炸。

原因:for-each 内部用迭代器(iterator),迭代器会检测"外部是否修改了集合结构"(modCount 计数器)。一旦检测到,就立刻抛异常——这是fail-fast 机制,为了避免并发修改导致数据错乱。

错误:

```
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
for (Integer x : list) {
    if (x % 2 == 0) list.remove(x); // 抛 ConcurrentModificationException
}
```

正确(三种,任选一种):

```
// 方式 1:用迭代器的 remove(推荐)
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    int x = it.next();
    if (x % 2 == 0) it.remove(); // 通过迭代器删
}

// 方式 2:倒着遍历下标(下标可控)
for (int i = list.size() - 1; i >= 0; i--) {
    if (list.get(i) % 2 == 0) list.remove(i);
}

// 方式 3:先收集要删的,遍历完再批删
List<Integer> toRemove = new ArrayList<>();
for (Integer x : list) {
    if (x % 2 == 0) toRemove.add(x);
}
list.removeAll(toRemove);
```

C++ 对照:C++ 范围 for + erase 也有类似的"迭代器失效"问题,但不会抛异常(会变成 UB,更危险)。

详见:02 语法差异速览 第十节"for-each 增强 for 循环"。

坑 5:`Arrays.asList` 返回定长 list()



表现:Arrays.asList(1, 2, 3).add(4) 抛 UnsupportedOperationException。

原因:Arrays.asList 返回的是 Arrays.ArrayList(在 Arrays 内部),不是`java.util.ArrayList`。这个内部类直接包装传入的数组,不支持 add / remove / clear 这类改变长度的操作。set(index, value) 改元素是允许的(不改变长度)。

错误:

```
List<Integer> list = Arrays.asList(1, 2, 3);
list.add(4);      // 抛 UnsupportedOperationException
list.remove(0);   // 同上
```

正确:

```
// 外面包一层 ArrayList,得到真正的可变 list
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3));
list.add(4);      // 正常
list.remove(0);   // 正常

// 另一个思路:用 Stream(Java 8+)
List<Integer> list2 = Stream.of(1, 2, 3).collect(Collectors.toList());
list2.add(4);     // 正常
```

详见:03 容器与 STL 对照 第四节"深讲 1:ArrayList" 坑点 2。

C++ 对照:C++ 没有这个坑,std::vector 初始化后就能 push_back。

坑 6:`String` 拼接性能(用`StringBuilder`)

表现:循环里大量字符串拼接(String s = ""; for (...) s += i;),代码 TLE(超时)。

原因:String 是不可变对象(见 02 第二节),每次 += 都会创建新 String 对象,循环 N 次就是 O(n²) 次对象创建。StringBuilder 是可变字符串,内部用 char[] 数组,append 是 O(1) 摊销,总复杂度 O(n)。

错误(性能差,100000 次循环实测慢 100+ 倍):

```
String s = "";
for (int i = 0; i < 100000; i++) {
    s += i;    // 每次 += 创建新 String 对象
}
```

正确:

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 100000; i++) {
    sb.append(i);
}
String s = sb.toString(); // 最后转成 String
```

详见:04 常用工具类 "StringBuilder" 小节(04 文档生成后引用)。

C++ 对照:C++ std::string += 是 O(1) 摊销(类似 StringBuilder),没有这个性能坑。但 C++ 选手写 ostringstream 也算等价方案。

坑 7:浮点精度(用`Math.abs(a-b) < eps`)

表现:0.1 + 0.2 == 0.3 返回 false(实际是 0.30000000000000004)。LeetCode 上"浮点返回值"类题目常因此 Wrong Answer。

原因:IEEE 754 浮点表示的固有误差,十进制小数无法精确用二进制表示(0.1 = 0.0001100110011... 无限循环)。



错误:

```
double a = 0.1 + 0.2;
if (a == 0.3) {      // false
    System.out.println("equal");
}
```

正确(两种,按场景选):

```
// 方式 1:用 eps 容差(刷题推荐,够用)
double a = 0.1 + 0.2;
if (Math.abs(a - 0.3) < 1e-9) {    // 1e-9 是常用 eps
    System.out.println("equal");    // 命中
}

// 方式 2:用 BigDecimal(精确,刷题不常用,工程用)
BigDecimal x = new BigDecimal("0.1").add(new BigDecimal("0.2"));
// x = 0.3(精确)
```

C++ 对照:C++ 同样有这个问题,处理方式完全相同。

```
// C++ 一样要用 eps
double a = 0.1 + 0.2;
if (std::abs(a - 0.3) < 1e-9) { /* ... */ }
```

坑 8:数组排序 + 自定义比较器(尤其 `int[]` 二维)

表现: `int[][] points` 想按第二个元素排序,直接 `Arrays.sort(points)` 按第一个排,提交 Wrong Answer。

原因: `Arrays.sort(int[][])` 默认按行首元素升序,因为 `int[]` 实现了 `Comparable`(按字典序比较数组)。想按其他键排序,必须显式传 `Comparator`。

错误:

```
int[][] points = {{1, 3}, {2, 1}, {3, 2}};
Arrays.sort(points);    // 按首元素排:[1,3], [2,1], [3,2] —— 不是要的顺序
```

正确(三种写法,按需取用):

```
int[][] points = {{1, 3}, {2, 1}, {3, 2}};

// 方式 1:lambda 按第二个元素升序(最常用)
Arrays.sort(points, (a, b) -> a[1] - b[1]);
// 结果:[2,1], [3,2], [1,3]

// 方式 2:按第二个元素降序
Arrays.sort(points, (a, b) -> b[1] - a[1]);

// 方式 3:多键排序(先按第二个,再按第一个),用 Comparator 链式
Arrays.sort(points, Comparator
    .comparingInt((int[] p) -> p[1])    // 主键:p[1]
    .thenComparingInt(p -> p[0]));    // 次键:p[0]
```

详见:04 常用工具类 "Comparator" 小节(链式调用 `comparingInt` / `thenComparingInt` 详讲)。

C++ 对照:

```
// C++ 用 lambda 一行
std::sort(points.begin(), points.end(), [](const auto& a, const auto& b) {
    return a[1] < b[1];
});
```



坑 9: `int[]` vs `Integer[]` 在 `Arrays.asList` 行为不同(进阶)

表现: `Arrays.asList(new int[]{1, 2, 3})` 编译通过但行为诡异; `Arrays.asList(new Integer[]{1, 2, 3})` 正常。

原因: `Arrays.asList(T... a)` 接受泛型 T 的可变参数。`Integer[]` 是 T = Integer 的数组,正常展开;`int[]` 不是 T(Java 泛型不能用基本类型),会被当成单个对象(整个 `int[]` 数组),所以 `asList` 返回的是 `List<int[]>`,里面就一个元素。

错误:

```
List<int[]> list = Arrays.asList(new int[]{1, 2, 3});
// list.size() == 1,不是 3!
list.get(0); // int[]{1, 2, 3} 这个数组本身
// 想要的是 List<Integer>,所以错了
```

正确(三种):

```
// 方式 1:用包装类 Integer[]
List<Integer> list1 = Arrays.asList(new Integer[]{1, 2, 3});
// 记得包 ArrayList 才能 add
List<Integer> mutable = new ArrayList<>(list1);

// 方式 2:用 Stream(Java 8+)+ 装箱
List<Integer> list2 = Arrays.stream(new int[]{1, 2, 3})
    .boxed()
    .collect(Collectors.toList());

// 方式 3:Java 9+ List.of(整型自动装箱)
List<Integer> list3 = List.of(1, 2, 3); // 但 List.of 同样不能 add
```

记忆口诀: `Arrays.asList(基本类型数组)` = 装整个数组; `Arrays.asList(包装类型数组)` = 装每个元素。

坑 10: `HashMap` 多线程(单线程够用)

表现:理论上,多线程下 `HashMap` 扩容可能死循环 / 数据丢失。刷题单线程碰不到,但面试会问。

原因:Java 8 之前,`HashMap` 扩容时用头插法把链表搬到新桶,多线程并发扩容可能让链表成环,`get` 时死循环。Java 8+ 改成了尾插法+ 红黑树,死循环问题解决,但多线程下仍可能数据丢失。

处理:

```
// 刷题(单线程):用 HashMap,完全没问题
Map<String, Integer> map = new HashMap<>();

// 多线程:用 ConcurrentHashMap
Map<String, Integer> safeMap = new ConcurrentHashMap<>();
// 用法跟 HashMap 几乎一样
```

C++ 对照:C++ `std::unordered\_map` 多线程也要加锁(或 `tbb::concurrent\_unordered\_map`),没 Java 区分得细。

坑 11: 【C++ 选手第 1 坑】 `String` 比较用 `==` 还是 `equals`()

表现: `String s1 = "abc"; String s2 = new String("abc"); s1 == s2` 返回 false,但内容明明一样。

原因: `==` 在 Java 中永远比较引用地址(对于对象类型)。`s1` 指向字符串常量池中的 "abc", `s2` 指向堆上新 new 出来的对象,两个对象地址不同,所以 `==` 返回 false。

错误(C++ 选手第一周必踩):



```
String s1 = "abc";
String s2 = new String("abc");
if (s1 == s2) {           // false!你以为内容相等
    System.out.println("equal");
}
```

正确:

```
String s1 = "abc";
String s2 = new String("abc");
if (s1.equals(s2)) {      // true,比内容
    System.out.println("equal");
}

// null 安全:把已知非空的放前面
if ("abc".equals(s2)) {   // s2 是 null 也不会崩
    System.out.println("equal");
}
```

详见:04 常用工具类 "String" 小节(详讲字符串常量池、intern 等概念)。

C++ 对照(反直觉):

```
// C++ std::string == 直接比内容
std::string s1 = "abc";
std::string s2 = "abc";
if (s1 == s2) {           // true,内容相等
    std::cout << "equal";
}
// C++ 的 == 是运算符重载,语义跟 Java 的 .equals() 一样
```

记忆口诀:Java `==` 比地址,内容比 `equals`;C++ `==` 比内容,反的!

坑 12: 【C++ 选手第 2 坑】数组长度 / 字符串长度 / 集合大小语法不同()

表现:arr.length()(报错)/ s.length(报错)/ list.size(报错)——C++ 选手第一周会写错至少 2 次。

原因:Java 历史上把"长度"这个概念分成了三种不同写法(数组用 length 字段、String 用 length() 方法、集合用 size() 方法),没统一。这是历史包袱,不是设计。

错误:

```
int[] arr = {1, 2, 3};
arr.length();           // 编译报错!arr.length 是字段,不是方法

String s = "hello";
int n = s.length;       // 编译报错!String 的 length 是方法

List<Integer> list = new ArrayList<>();
int sz = list.size;     // 编译报错!Collection 的 size 是方法
```

正确:



```
int[] arr = {1, 2, 3};
int n1 = arr.length;    // 字段(无括号)

String s = "hello";
int n2 = s.length();    // 方法(有括号)

List<Integer> list = new ArrayList<>();
int sz = list.size();    // 方法(有括号)

// 二维数组用 .length 取行数,每行用 .length 取列数
int[][] grid = {{1, 2}, {3, 4, 5}};
int rows = grid.length;    // 2(行数)
int cols = grid[0].length; // 2(第 0 行列数,各行可能不同)
```

C++ 对照(反直觉,Java 这种混乱会让 C++ 选手懵):

```
// C++ 全是 .size(),无脑一致
int arr[] = {1, 2, 3};
size_t n1 = std::size(arr); // 或 sizeof(arr)/sizeof(arr[0])

std::string s = "hello";
size_t n2 = s.size();

std::vector<int> v;
size_t sz = v.size();
```

记忆口诀:数组 `length` 字段(无括号),字符串 `length()` 方法(有括号),集合 `size()` 方法(有括号)。

坑 13:Java 数组默认初始化()【C++ 选手必看】

表现: `int[] a = new int[5];` 元素全 0 / `boolean[] b = new boolean[5];` 全 false / `String[] s = new String[5];` 全 null。C++ 选手会担心"是不是垃圾值?"。

原因:Java 数组在 new 时自动初始化——基本类型元素初始化为 0 / false,引用类型元素初始化为 null。没有未初始化状态,绝对不会有"垃圾值"。

错误理解:

```
int[] a = new int[5];
// C++ 选手的担心:a[0] 是垃圾值?会 UB?
// 实际:Java 保证 a[0] == 0,放心用
```

正确理解:

```
int[] a = new int[5];
// a = [0, 0, 0, 0, 0],保证初始化为 0

boolean[] b = new boolean[5];
// b = [false, false, false, false, false]

String[] s = new String[5];
// s = [null, null, null, null, null] ← 注意是 null,不是空串 ""
```

C++ 对照(反直觉,C++ 行为完全不同):

```
int a[5];
// C++ 数组元素是垃圾值(未定义行为),可能读到任意内存!
// 必须手动初始化:int a[5] = {}; 或 std::fill(a, a+5, 0);

std::string s[5];
// s[0] 默认构造空串 "",Java 则是 null(差异!)
```



关键提醒:Java 数组元素的 `null` 跟"空串"是两回事。遍历字符串数组时,先检查 `s[i] != null` 再说。

坑 14:递归栈深度限制(`StackOverflowError`)() 【C++ 选手必看】

表现:深链表(10000+ 节点)反转 / 深二叉树用递归,Java 抛 `StackOverflowError`,C++ 一般不抛。

原因:JVM 默认线程栈大小远小于 C++(通常 512KB - 1MB,看 OS),递归调用每层消耗栈空间,深一点就爆栈。C++ 默认栈 8MB,LeetCode 数据范围内一般不会爆。

错误(深链表反转用递归,容易爆栈):

```
public ListNode reverseList(ListNode head) {
    if (head == null || head.next == null) return head;
    ListNode newHead = reverseList(head.next); // 递归
    head.next.next = head;
    head.next = null;
    return newHead;
}
// 链表长度 50000+ 时抛 StackOverflowError
```

正确(改迭代):

```
public ListNode reverseList(ListNode head) {
    ListNode prev = null, cur = head;
    while (cur != null) {
        ListNode nxt = cur.next;
        cur.next = prev;
        prev = cur;
        cur = nxt;
    }
    return prev; // 迭代,无栈深度问题
}
```

进阶(真要递归时):增加栈大小 `java -Xss16m Main`,但刷题场景直接改迭代最稳。

C++ 对照(C++ 选手常踩,因为 C++ 不爆):

```
// C++ 默认栈 8MB,深递归一般不爆Java 选手常"借鉴"过来踩坑
ListNode* reverseList(ListNode* head) {
    if (!head || !head->next) return head;
    ListNode* newHead = reverseList(head->next); // C++ 能跑,Java 炸
    head->next->next = head;
    head->next = nullptr;
    return newHead;
}
```

详见:01 输入输出篇 模板 5"LeetCode 链表输入"(那个迭代版本是稳的)。

记忆口诀:深递归爆栈,改迭代;二叉树 DFS 也优先迭代。

坑 15:ACM 模式 `main` 函数必须 `throws IOException` ()

表现:`BufferedReader br = new BufferedReader(new InputStreamReader(System.in));` 编译报错 `unhandled exception: IOException`。

原因:Java 把"可能出错的 I/O 操作"标成受检异常(`checked exception`),`BufferedReader.readLine()` 抛 `IOException`,编译器强制你处理。要么 `throws`,要么 `try-catch`,不写就编译失败。

错误:



```
public class Main {
    public static void main(String[] args) { // 编译失败!
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        String line = br.readLine(); // IOException 未处理
    }
}
```

正确:

```
public class Main {
    public static void main(String[] args) throws IOException { // ★ 必加
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        String line = br.readLine();
        System.out.println(line);
    }
}
```

详见:01 输入输出篇 第六节"异常声明 throws IOException"。

C++ 对照(C++ 无此概念):

```
int main() {
    // C++ 没 checked exception,不写也能跑
    std::string line;
    std::getline(std::cin, line);
    std::cout << line;
    return 0;
}
```

记忆口诀:ACM 模式 `main` 必加 `throws IOException`,LeetCode 模式不用加。

三、避坑金句速记(每天刷题前过一遍)

1. Stack → ArrayDeque(别用 java.util.Stack)
2. PriorityQueue 默认小顶堆(反 C++ 要 Comparator.reverseOrder())
3. Integer / Long / String 比内容用 equals (缓存池陷阱)
4. 遍历时不能 remove / put (用迭代器)
5. Arrays.asList 不能 add / remove (包一层 ArrayList)
6. String 拼接用 StringBuilder (+ = 是 $O(n^2)$)
7. 浮点比较用 $\text{Math.abs}(a-b) < \text{eps}$ ($\text{eps} = 1e-9$)
8. 一维数组排序要传 Comparator
9. int[] vs Integer[] 在 asList 行为不同(基本类型当一个对象)
10. HashMap 单线程够用,多线程用 ConcurrentHashMap
11. String == 比地址,内容比 equals (反 C++)
12. length (数组字段) / length() (String) / size() (集合) 三种写法(反 C++)
13. Java 数组默认 0 / false / null (C++ 是垃圾值,反)
14. 深递归爆栈,改迭代(C++ 默认栈大,Java 小)
15. ACM main 必加 throws IOException (LeetCode 不用)

四、15 大坑快速对照表

#	坑点	必踩度	C++ 选手必看	详见
1	`Stack` 改用 `ArrayDeque`	★★★		03 第五节
2	`PriorityQueue` 默认小顶堆	★★★		03 第六节
3	`Integer ==` 缓存池	★★★	★	03 第三节装箱小框



4	遍历时 `remove` 抛异常	★★		02 第十节
5	`Arrays.asList` 不能 `add`	★★		03 第四节
6	`String` 拼接性能	★★		04 StringBuilder
7	浮点精度用 eps	★★		本篇
8	数组排序 + `Comparator`	★★		04 Comparator
9	`int[]` vs `Integer[]`	★		本篇
10	`HashMap` 多线程	★		本篇
11	`String ==` 比地址	★★★★	★	04 String
12	`length` / `length()` / `size()`	★★★★	★	02 第三节
13	数组默认初始化 0/null	★★	★	02 第三节
14	深递归 `StackOverflowError`	★★	★	01 模板 5
15	ACM `main` 必加 `throws`	★★★★		01 第六节

练习建议

- 抄一遍 15 大坑的错误 / 正确代码,每个 5 分钟,共 1.5 小时
- 在自己的 IDE 里跑一下坑 2(PriorityQueue)/ 坑 3(Integer ==)/ 坑 11(String ==),看实际输出,加深印象
- 找 1 道递归题(LeetCode 206 链表反转 / 104 二叉树最大深度),先用递归写一遍看会不会爆栈,再改迭代,体会"为什么 Java 要慎用递归"
- 每次刷题前扫一遍"避坑金句速记",直到能背
- 做题时遇到 ConcurrentModificationException / UnsupportedOperationException / NullPointerException,第一时间翻本章对应坑

下一步

打开 07_实战例题.md,7 道典型题完整流程(从输入到提交),把上面 8 篇所有知识串起来。

